

Transformer

Hyoungh Joon, Kim

June 3, 2026

Contents

1	Introduction	2
2	Main Ideas	2
2.1	Attention	2
2.1.1	Dot-product Self-attention	2
2.1.2	Intuition of the Attention Mechanism	3
3	Extensions of Attention	4
3.1	Scaled Dot-product Self-attention	4
3.2	Multi-head Attention	5
3.3	Positional Encoding	5

1 Introduction

The **transformer** architecture was originally introduced for NLP (Natural Language Processing) , and surpasses the RNN (Recurrent Neural Network), which was the mainstream of NLP. Transformers can process various types of data, even if it was developed for NLP. For example, ViT (vision transformers) often outperform CNN in image processing tasks.

Transformers has following merits:

- Effective in transfer learning: The transformer can be trained on large dataset, and then can be fine-tuned for downstream tasks.
- Well-suited to massive parallel processors such as GPUs. This accelerates training and inference process of transformers, which RNN cannot do.
- Shares parameters, which leads to more efficient memory usage than ordinary full-connected layer.

2 Main Ideas

2.1 Attention

The transformer is based on the **attention** mechanism. The attention calculates how much the data must take *attention* to other data. For example, consider following two sentences:

I swam across the river to get to the bank.

I walked across the road to get cash from the bank.

The word *bank* has different interpretation in two sentences, and this difference can be found from other words in each sentence:

- The meaning of the bank in first sentence can be induced from *swam* and *river*.
- The meaning of the bank in second sentence can be induced from *cash*.

So, to process this sentences, one must estimate the amount of *attention* that words take on each other words.

2.1.1 Dot-product Self-attention

Now we discuss the precise structure of the heart of the transformer architecture, the attention mechanism.

Definition 2.1.1 (Dot-product Self-attention)

Let $\mathbf{X}$ be $N \times D$ matrix, which consists of N $D \times 1$ data vectors. Then, the **dot-product self-attention** $\mathbf{Sa}$, which is an $N \times D_v$ matrix, is defined as following:

$$\mathbf{Sa}[\mathbf{X}] = \text{Softmax}(\mathbf{Q}\mathbf{K}^\top) \mathbf{V}$$

where $\mathbf{V}$, $\mathbf{Q}$, $\mathbf{K}$, called value, query and key, is defined as following:

$$\mathbf{V} = \mathbf{1}\beta_v^\top + \mathbf{X}\mathbf{W}_v^\top$$

$$\mathbf{Q} = \mathbf{1}\beta_q^\top + \mathbf{X}\mathbf{W}_q^\top$$

$$\mathbf{K} = \mathbf{1}\beta_k^\top + \mathbf{X}\mathbf{W}_k^\top$$

- $\mathbf{W}_v, \beta_v$ have dimensionality $D_v \times D$ and $D_v \times 1$, respectively.
- $\mathbf{W}_q, \beta_q$ have dimensionality $D_q \times D$ and $D_q \times 1$, respectively.
- $\mathbf{W}_k, \beta_k$ have dimensionality $D_k \times D$ and $D_k \times 1$, respectively.

$\mathbf{1}$ is an $N \times 1$ vector. D_v governs the size of the output vectors. The **Softmax** function applies ordinary softmax function to each row, respectively.

The key represents the information which the value is holding. The query represents what value is asking to other values, required to understand its meaning in the whole data. If the query and key matches well then the dot product will produce large value. If not, the dot product result will be small. This is the 'attention'.

This is quite unintuitive at first look, so let's get some intuition of these equations.

2.1.2 Intuition of the Attention Mechanism

The motivation of the attention is to calculate the amount of attention the data pays to every other data, which can be expressed as following:

$$\mathbf{y}_n = \sum_{m=1}^N a_{nm} \mathbf{x}_m$$

with data $\{\mathbf{x}_1 \dots \mathbf{x}_N\}$. The a_{nm} is *attention weight*, and we can interpret this as how much the $\mathbf{x}_n$ pay attention to $\mathbf{x}_m$. If a_{nm} is high, then $\mathbf{x}_n$ is highly related, or pays attention, to $\mathbf{x}_m$. If it is not, then the relevancy is low. To let the a_{nm} represent the amount of attention in reasonable way, we would like to constrain the attention weights in the following way:

$$\begin{aligned} a_{nm} &\geq 0, \\ \sum_{m=1}^N a_{nm} &= 1. \end{aligned}$$

Now, one of the simple way of calculating a_{nm} is to take the dot product of $\mathbf{x}_n$ and $\mathbf{x}_m$. But, we can make it more flexible by introducing the **query** and **key** vectors;

$$\begin{aligned} \mathbf{q}_n &= \beta_q + \mathbf{W}_q \mathbf{x}_n \\ \mathbf{k}_m &= \beta_k + \mathbf{W}_k \mathbf{x}_m \end{aligned}$$

Now we must calculate how much the query should pay attention to the key value. We use softmax function to meet the requirements we considered earlier;

$$a_{nm} = \frac{\exp \mathbf{q}_n^\top \mathbf{k}_m}{\sum_{i=1}^N \exp \mathbf{q}_n^\top \mathbf{k}_i}$$

Now we transform the vector $\mathbf{x}_m$ to $\mathbf{v}_m$, by following equation:

$$\mathbf{v}_m = \beta_v + \mathbf{W}_v \mathbf{x}_m$$

where β_v and $\mathbf{W}_v$ are bias and weights, respectively. Since the original data can be difficult to directly recognize the relevance, we are transforming it.

So, the final output is

$$\mathbf{y}_n = \sum_{m=1}^N \frac{\exp \mathbf{q}_n^\top \mathbf{k}_m}{\sum_{i=1}^N \exp \mathbf{q}_n^\top \mathbf{k}_i} \mathbf{v}_m$$

Remark 2.1.1 (Summary of the meaning of each matrices)

To summarize, the following are the meaning of each matrices:

- $\mathbf{q}_n$: The question that $\mathbf{v}_n$ asks to other key.
- $\mathbf{k}_n$: The information that $\mathbf{v}_n$ is holding.
- the attention weight $a_{nm} = \frac{\exp \mathbf{q}_n^\top \mathbf{k}_m}{\sum_{i=1}^N \exp \mathbf{q}_n^\top \mathbf{k}_i}$: the amount of attention that $\mathbf{v}_n$ is paying to $\mathbf{v}_m$.
- $\mathbf{y}_n = \sum_{m=1}^N \frac{\exp \mathbf{q}_n^\top \mathbf{k}_m}{\sum_{i=1}^N \exp \mathbf{q}_n^\top \mathbf{k}_i} \mathbf{v}_m$: Newly created context which is easier to interpret. The amount of attention that $\mathbf{v}_n$ is paying to other data is compressed in.

TODO

Remark 2.1.2

Added to ease understanding of the matrix notation of attention mechanism :

3 Extensions of Attention

The following extensions are used very often when using attention mechanism.

3.1 Scaled Dot-product Self-attention

The dot product in attention mechanism can have large magnitude, and the softmax function will going to produce very small gradient value for that dot product result. So we resize the value that softmax function receives by dividing with square root of the dimension of query vector (which must be same as key vector's dimension);

Definition 3.1.1 (Scaled Dot-product Self-attention)

Let D_k be the dimension of query vector (which equals key vector). Then the scaled dot-product self-attention is given as

$$\text{Sa}[\mathbf{X}] = \text{Softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{D_k}} \right) \mathbf{V}$$

This is called **scaled dot-product self-attention**.

3.2 Multi-head Attention

The self-attention mechanism make use of same parameter for each input data. This may decrease the expressive power of the attention, and does not consider the situation where various context must be used. To prevent this, we use multiple value, query, and key weights. We produce attention result in parallel with each weights and then concatenate them. This is called **multi-head attention**.

Definition 3.2.1 (Multi-head Attention)

Let X be an $N \times D$ matrix. Let $\mathbf{W}_v^h \in \mathbb{R}^{D_v \times D}$, $\mathbf{W}_q^h \in \mathbb{R}^{D_k \times D}$, and $\mathbf{W}_k^h \in \mathbb{R}^{D_k \times D}$ be the weights of h -th attention mechanism or **head**. Let $\beta_v^h \in \mathbb{R}^{D_v}$, $\beta_q^h \in \mathbb{R}^{D_k}$, and $\beta_k^h \in \mathbb{R}^{D_k}$ be the biases of h -th head. We calculate h -th value, query, and key as:

$$\begin{aligned}\mathbf{V}_h &= \mathbf{1}(\beta_v^h)^\top + \mathbf{X}(\mathbf{W}_v^h)^\top \\ \mathbf{Q}_h &= \mathbf{1}(\beta_q^h)^\top + \mathbf{X}(\mathbf{W}_q^h)^\top \\ \mathbf{K}_h &= \mathbf{1}(\beta_k^h)^\top + \mathbf{X}(\mathbf{W}_k^h)^\top\end{aligned}$$

Then, each head can be written as:

$$\text{Sa}_h[\mathbf{X}] = \text{Softmax} \left(\frac{\mathbf{Q}_h \mathbf{K}_h^\top}{\sqrt{D_k^h}} \right) \mathbf{V}_h$$

The multi-head attention is defined as follows;

$$\text{MhSA}[\mathbf{X}] = [\text{Sa}_1[\mathbf{X}], \dots, \text{Sa}_H[\mathbf{X}]] \mathbf{W}$$

where $\mathbf{W}$ is another weight matrix, and H is the number of heads.

If the number of heads is H , each D_v is usually chosen to be equal to D/H .

3.3 Positional Encoding

As the structure of the attention mechanism shows, the attention mechanism itself is equivariant to permutation of input values. However, the sequential information is important in NLP. To overcome this, we can inject a positional information into the input itself, without modifying the attention mechanism. This can be described as

$$\tilde{\mathbf{x}}_n = \mathbf{x}_n + \mathbf{r}_n$$

where $\mathbf{r}_n$ is a given positional encoding.

The following is a technique of positional encoding that uses sinusoidal functions.

Definition 3.3.1 (Sinusoidal Positional Encoding)

For a given position n , the position-encoding vector has components r_{ni} given by

$$r_{ni} = \begin{cases} \sin\left(\frac{n}{L^{i/D}}\right) & \text{if } i \text{ is even,} \\ \cos\left(\frac{n}{L^{(i-1)/D}}\right) & \text{if } i \text{ is odd} \end{cases}$$

This encoding can be thought of a continuous version of how binary numbers are represented. The lower the digit, the higher its frequency.